Northeastern University

Mechanical and Industrial Engineering Department

OR 7230: Probabilistic Operations Research

Prof. Jin

Spring 2026

Probabilistic Inventory Optimization for Perishable Food Items

Final Report

Aryan Anand Gupta

Contents

1 I. Introduction & Objective **1**

2 II. Data Collection & Processing **1**

2.1 Data Source . . . 1

2.2 Cleaning . . . 1

2.3 Demand Distribution . . . 1

2.4 Validation: Pooled vs. Weekday-Specific . . . 2

3 III. Model Design **3**

3.1 System Setup . . . 3

3.2 Why Track Freshness? . . . 3

3.3 Daily Sequence . . . 3

3.4 Cost Objective . . . 3

3.5 MDP Formulation . . . 4

4 IV. Algorithms & Implementation **4**

4.1 Monte Carlo Simulator . . . 4

4.2 Grid Search for Best Static Policy . . . 4

4.3 MDP Value Iteration . . . 4

5 V. Results & Discussion **4**

5.1 Why Simple Rules Fail . . . 4

5.2 Optimal Static Policy . . . 5

5.3 MDP Policy . . . 6

5.4 Sensitivity Analysis . . . 7

5.4.1 Cost Parameters . . . 7

5.4.2 Lead Time Variability . . . 7

5.4.3 Shelf Life . . . 8

5.5 The Unifying Insight . . . 9

6 VI. Validation **9**

7 VII. Conclusions **10**

References **11**

Appendix A: Raw Data **12**

Appendix B: Code **13**

I. Introduction & Objective

At a local coffee shop, one of their most popular sandwiches was frequently out of stock despite what seemed like consistent customer interest. People would walk in, ask for it, and leave without buying anything. The shop was losing sales, but if they ordered too many, unsold sandwiches would end up in the trash by the end of the day. This is a classic operations research problem: how much should you order each day when demand is uncertain and the product is perishable?

This project develops a simulation-based framework to find a cost-minimizing daily ordering policy for that sandwich. Specifically, the goals are:

1. Estimate a demand distribution from real transaction data and validate that it generalizes.
2. Test simple ordering heuristics (ordering the mean, the newsvendor formula) and understand why they perform poorly in this setting.
3. Use simulation-based grid search to find the best fixed ordering rule.
4. Formulate and solve a Markov Decision Process (MDP) that adapts orders to the current shelf state.
5. Run sensitivity analysis across cost structures, delivery reliability, and shelf life.

The entire pipeline is implemented in Python using Jupyter notebooks. For the MDP solver, which is computationally intensive, I wrote a C extension that gets compiled at runtime for speed.

II. Data Collection & Processing

Data Source

The project uses a publicly available coffee shop transaction dataset from Kaggle, covering roughly one year of sales (January–December 2023). There are 9,741 individual transactions with fields for item name, quantity, price, and date. Raw data is included in the appendix.

Cleaning

I filtered out all sandwich transactions and aggregated them into daily totals. For days with no sandwich sales at all, I filled in a zero, since no sales on a given day still counts as observing zero demand. One day (February 6) had an anomalous spike that was inconsistent with the surrounding days, so it was removed as an outlier. After cleaning, the dataset has **364 daily demand observations**.

Demand Distribution

From these 364 observations, I built a probability mass function (PMF) by counting how often each demand level (0 through 25) appeared. The issue is that some demand levels were never observed, which would cause the model to assign them zero probability, effectively treating those levels as impossible. To handle this, I applied a standard smoothing technique where one pseudo-count is added to each level before computing the fractions. The overall shape of the distribution stays the

same, but now every demand level has some nonzero probability.

The resulting distribution has a mean of about 8.96, median of 8, standard deviation around 6.47, and a coefficient of variation of 0.72. It is noticeably right-skewed with a long tail stretching out to 25. As it turns out, this skewness is the main reason why simple ordering rules perform so poorly in this system.

Figure 1 shows the distribution visually. The peak sits around 6 to 8 sandwiches per day, which is where the most common demand levels cluster. But notice how the right tail extends all the way out to 25, with meaningful probability mass spread across 10 to 20. On any given day, there is roughly a 25% chance that demand exceeds 14. This means that ordering near the mean (about 9) leaves the shop unprepared for a large fraction of possible outcomes. The gap between the median (8) and the 95th percentile (19) is what drives most of the difficulty in this problem.

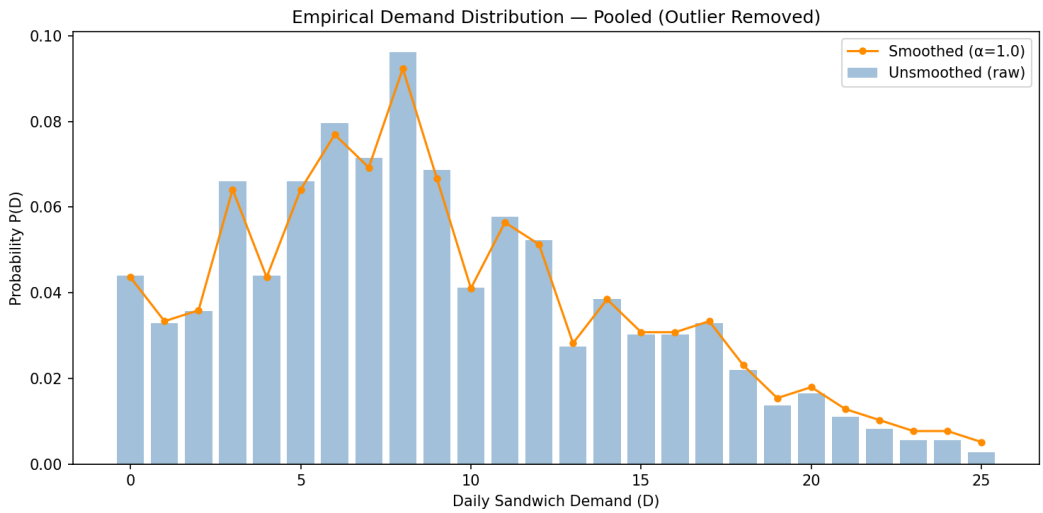

Figure 1: Empirical demand distribution (bars) with the smoothed PMF overlaid (line). Demand peaks around 6–8 sandwiches per day but has a long right tail out to 25, resulting in a coefficient of variation of 0.72.

Validation: Pooled vs. Weekday-Specific

One concern was whether demand patterns differ by day of week. If they do, we would need seven separate distributions instead of one pooled model. To check, I split the data by weekday (≈52 observations each) and fit separate PMFs. Then I used a train/test split to see which approach predicted held-out data better. Somewhat surprisingly, the single pooled distribution outperformed the weekday-specific ones (average test log-likelihood: -3.04 pooled vs. -3.15 weekday). With only about 52 observations per weekday, the per-day models were overfitting to noise rather than capturing real patterns. Based on this, I used the single pooled distribution for all subsequent modeling.

III. Model Design

System Setup

The system works as follows. Each morning, the café receives any orders that were placed previously (delivery takes either 1 or 2 days), then decides how many new sandwiches to order. Throughout the day, customers arrive and sandwiches are sold. Unsold sandwiches remain on the shelf and age; after 3 days, anything left over gets thrown away. If a customer wants a sandwich and there are none available, that sale is lost; we assume they don't come back.

Table 1: System Parameters

Parameter	Value
Shelf life	3 days (oldest consumed first)
Delivery lead time	1 day (prob. 0.8) or 2 days (prob. 0.2)
Demand	Smoothed empirical PMF, $D = 0, \ldots, 25$
Sell price / Unit cost / Salvage	\$4.00 / \$1.50 / \$0.00
Stockout cost (c_u)	\$2.50 per missed sale
Waste cost (c_o)	\$1.50 per expired sandwich

Why Track Freshness?

A critical modeling decision was to track not just the total number of sandwiches on hand, but their ages. Consider two scenarios: the shop has 10 sandwiches that were made this morning, versus 10 sandwiches that expire at the end of today. In the first case, most will probably get sold over the next couple of days. In the second case, most are headed for the trash. A model that only tracks the total count would treat these identically, which clearly leads to bad ordering decisions.

To capture this, the inventory state is represented as (E_0, E_1, E_2, A_1):

- E_0 = units expiring today (oldest, eaten first)
- E_1 = units expiring tomorrow
- E_2 = freshest units on hand (2 days left)
- A_1 = units in the delivery pipeline arriving tomorrow

Daily Sequence

Each day follows five steps: (1) receive deliveries, (2) decide order quantity, (3) observe demand and serve customers oldest-first, (4) discard any units that expired, (5) age remaining inventory by one day.

Cost Objective

The daily cost is: stockout cost (\$2.50$\times$ unmet demand) + waste cost (\$1.50$\times$ expired units). The goal is to minimize the long-run average of this daily cost.

MDP Formulation

This naturally leads to a Markov Decision Process formulation. The state is (E_0, E_1, E_2, A_1), the action is how many sandwiches to order, and the transitions depend on random demand and random lead time. The goal is to find a policy that maps every possible shelf state to the best order quantity. I set a capacity cap of 35 units, which gives roughly 82,000 feasible states. The optimal policy is computed via value iteration with a discount factor of 0.99.

IV. Algorithms & Implementation

Monte Carlo Simulator

All policy evaluations use a vectorized Monte Carlo simulator that I wrote in NumPy. It runs 1,000 independent simulations of 364 days each, all simultaneously through array operations rather than looping over individual runs. A single policy evaluation takes about 0.05 seconds, which made it practical to sweep over dozens of candidate policies during grid search.

Grid Search for Best Static Policy

The simplest class of policies is "order-up-to S": each day, check how much inventory is in the system (on the shelf plus in the pipeline), and order enough to bring the total up to S. I swept S from 1 to 50, running the full 1,000-run Monte Carlo simulation at each level, and picked the S that minimized expected daily cost.

MDP Value Iteration

With about 82,000 states, value iteration requires sweeping over the entire state space repeatedly until convergence. Doing this in pure Python was too slow (around 72 seconds per sweep), so I wrote the inner Bellman update loop in C and compiled it at runtime with aggressive compiler optimizations. This brought each sweep down to about 0.2 seconds, a 380$\times$ speedup. The algorithm converges in roughly 21 iterations, so the total solve time is around 5 seconds. One implementation detail that turned out to be important was using tight loop bounds that skip infeasible states rather than iterating over the full grid and checking feasibility inside the loop.

V. Results & Discussion

Why Simple Rules Fail

I started by testing two standard textbook heuristics:

- **Mean policy ($S = 9$)**: order up to average demand. This achieved a fill rate of only 40%.
- **Newsvendor ($S = 10$)**: order to the critical fractile $c_u/(c_u + c_o) = 0.625$. Fill rate: 44%.

These results are quite poor. Roughly 60% of customer demand goes unmet under either policy. Looking at Figure 1 makes it clear why. The demand distribution is heavily right-skewed: the

mean sits around 9, but on many days demand reaches 15, 20, or even 25. Ordering 9 or 10 sandwiches means the shop runs out on essentially any above-average day. The newsvendor formula only gives $S = 10$ here because the critical fractile of 0.625 maps to the 62.5th percentile of this particular distribution, which happens to land at just one unit above the mean.

Optimal Static Policy

The grid search identified $S^* = 33$ as the cost-minimizing order-up-to level, with an expected cost of \$3.07/day and a 94.1% fill rate (Figure 2). This is a dramatic improvement. Cost drops by 78% compared to the mean policy, and the fill rate more than doubles. The reason S^* lands so far above the newsvendor's $S = 10$ is that this is not a single-period problem. The multi-day shelf life and stochastic lead time mean the system needs to carry substantially more buffer stock than a one-shot newsvendor model would suggest.

Figure 2 illustrates the full tradeoff. Panel (a) shows the U-shaped cost curve with S^* at the bottom: below 20, the shop constantly runs out; above 33, waste accumulates. Panel (b) decomposes this into falling stockout costs and rising waste costs. Panel (c) traces fill rate from near zero to above 95% around $S = 30$. Panel (d) shows that beyond $S = 28$ the shop wastes more per day than it fails to sell.

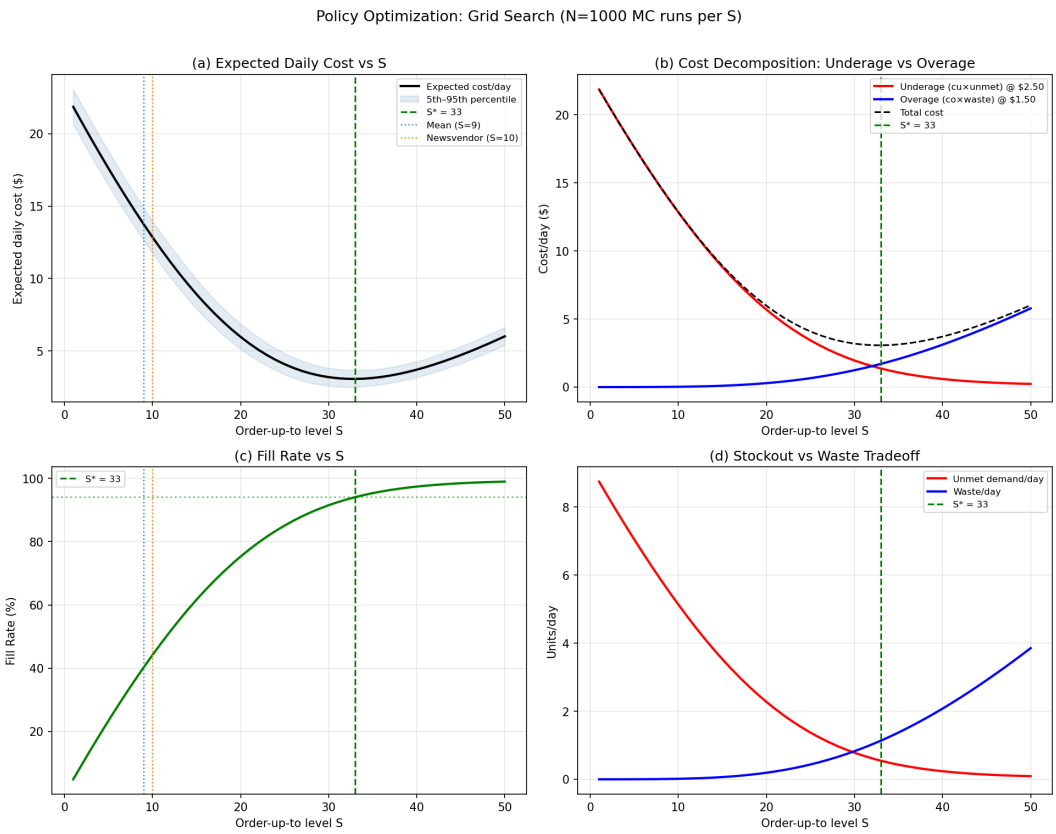

Figure 2: Policy optimization via grid search over $S = 1$ to 50. Panel (a) shows the U-shaped total cost with $S^* = 33$ marked. Panel (b) decomposes cost into stockout and waste components. Panel (c) shows fill rate vs. S. Panel (d) shows the stockout-waste tradeoff.

MDP Policy

The MDP policy brings the cost down further to \$2.68/day with a 94.4% fill rate, which is a **12.8% cost reduction** over the best static policy. What's interesting is that the fill rates are almost identical between the two. The MDP is not serving more customers. The improvement comes almost entirely from **waste reduction**: the MDP wastes 0.92 units/day compared to 1.14 for the static policy.

The reason is straightforward. A static order-up-to policy sees "15 sandwiches on the shelf" and decides not to order. But it doesn't know whether those 15 are fresh or about to expire. The MDP, on the other hand, might see "10 expiring today and 5 fresh" and recognize that most of those 10 will be thrown out regardless. So it orders fresh replacements for tomorrow while the static policy sits idle. The net effect is the same number of customers served, but fewer sandwiches in the trash.

Figure 3 visualizes how the MDP policy adapts to inventory freshness. The heatmap shows the optimal order quantity as a function of expiring stock (E_0, y-axis) versus fresh stock (E_2, x-axis), with E_1 and A_1 held at zero. The color gradient tells the story clearly: in the top-left region, where most inventory is old and about to expire, the MDP orders aggressively (15–20 units), because it knows that stock will likely go to waste regardless of demand. In the bottom-right region, where inventory is mostly fresh, the MDP holds back and orders very little (0–5 units), since those sandwiches still have days of shelf life left.

To put concrete numbers on it: if the shop has 15 sandwiches all expiring today and nothing fresh, the MDP orders about 20 more. But if those same 15 sandwiches are all fresh (2 days of life remaining), it orders closer to 15. A static policy cannot make this distinction at all. It sees "15 on the shelf" in both cases and makes the same decision.

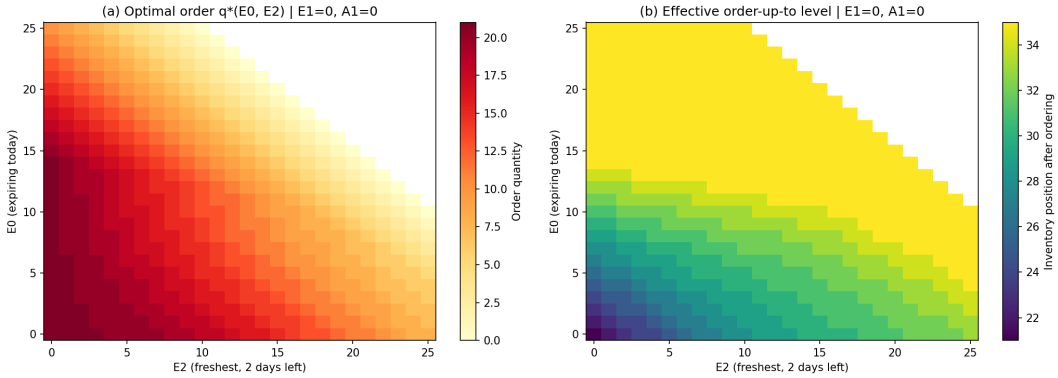

Figure 3: MDP optimal order quantity q^* as a function of expiring stock (E_0) and fresh stock (E_2), with $E_1 = 0$ and $A_1 = 0$. The MDP orders aggressively when inventory is mostly old (top-left) and conservatively when inventory is mostly fresh (bottom-right). A static policy would show a single uniform color across the entire plot.

Table 2: Policy Comparison (Base Case)

Policy	Cost/day	Fill Rate	Waste/day	Unmet/day
Mean ($S = 9$)	\$13.75	40.2%	0.01	5.49
Newsvendor ($S = 10$)	\$12.86	44.2%	0.02	5.13
Optimal Static ($S^* = 33$)	\$3.07	94.1%	1.14	0.55
MDP Policy	**\$2.68**	**94.4%**	**0.92**	**0.52**

Sensitivity Analysis

A natural follow-up question is: when is the MDP actually worth the added complexity? To explore this, I ran sensitivity analysis along three dimensions: cost structure, delivery reliability, and shelf life.

Cost Parameters

I tested four cost scenarios by varying the stockout and waste cost parameters. The MDP's advantage over the static policy ranged from 10% (when stockouts are very expensive) up to 14% (when waste is expensive). This makes sense: when stockout cost dominates, both policies converge on the same answer, which is to order aggressively. There is not much room for the MDP to do anything differently. But when waste is expensive, the static policy faces a dilemma: it cannot distinguish fresh inventory from expiring inventory, so it either overstocks and pays a high waste penalty, or understocks and loses sales. The MDP sidesteps this by observing what is about to expire and ordering accordingly.

Figure 4 shows the trend: the cost gap is widest in the "expensive waste" case, and the MDP's edge comes from consistently lower waste.

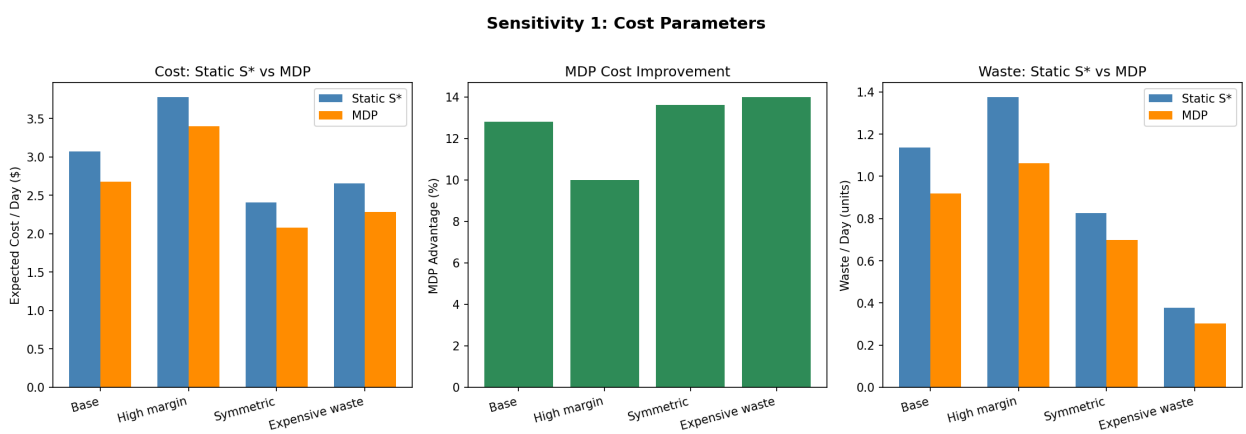

Figure 4: Cost sensitivity: MDP advantage ranges from 10% (high margin) to 14% (expensive waste). The MDP's edge comes from waste reduction, which is most valuable when waste is costly.

Lead Time Variability

Under deterministic delivery ($P(L{=}1) = 1.0$), the MDP advantage drops to 9.3%. This is expected: when the lead time is known with certainty, the pipeline state A_1 carries no useful information,

and the MDP's only edge comes from freshness awareness. The advantage is highest at 12.8% under the base case ($P(L{=}1) = 0.8$), where the MDP can also leverage pipeline information to avoid unnecessary reordering. At very high lead time noise ($P = 0.5$), the static policy partially compensates by raising S^* to 35, which narrows the gap somewhat.

Figure 5 confirms that both policies get more expensive as delivery becomes less reliable, but the MDP stays consistently below the static line through lower waste.

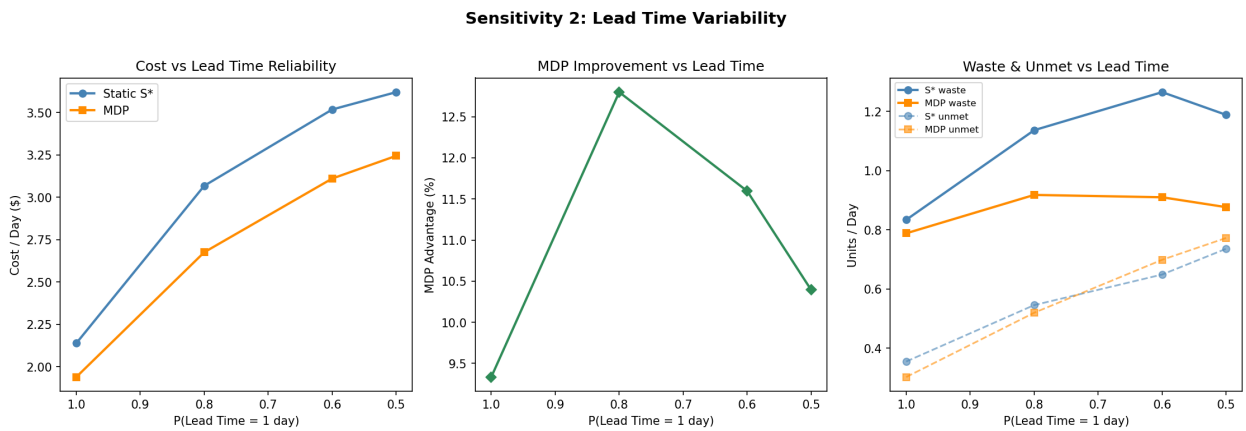

Figure 5: Lead time sensitivity: MDP advantage peaks at moderate variability ($P = 0.8$). At deterministic delivery, the pipeline state carries no information. At high noise, the static policy buffers by raising S^*.

Shelf Life

Shelf life turned out to be the single most impactful factor. When shelf life is reduced to 2 days, the MDP advantage jumps to **20.3%**, and it also improves the fill rate from 85.7% to 89.2%. With a shorter shelf life, sandwiches expire much faster, so knowing the age composition of the current inventory becomes much more valuable. At the other extreme, with a 4-day shelf life, almost nothing expires before getting sold, so there is essentially nothing for the MDP to optimize. I was unable to run the MDP for the 4-day case because the state space at the required capacity would have been around 90 million states, which is computationally intractable. However, an unconstrained grid search found $S^* = 37$ at \$1.61/day with near-zero waste, confirming that the MDP advantage would be negligible.

Figure 6 shows the trend: at 2-day shelf life the static policy costs \$6.40/day versus \$5.10 for the MDP, and the gap narrows steadily as shelf life increases.

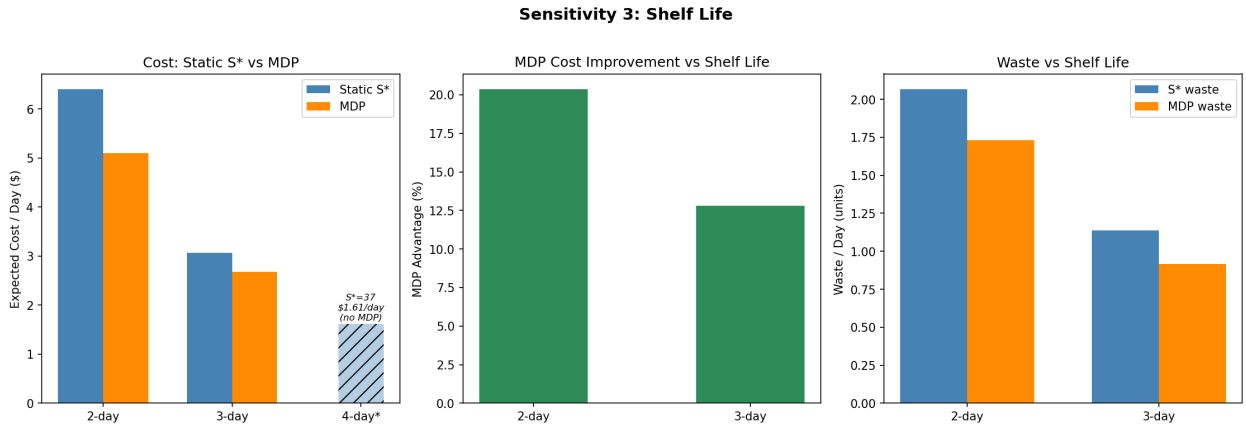

Figure 6: Shelf life sensitivity: MDP advantage drops from 20.3% (2-day) to 12.8% (3-day base) to ≈0% (4-day). The hatched 4-day bar shows the static-only result; MDP was intractable at required capacity.

Table 3: Sensitivity Summary: MDP Advantage Across Conditions

Dimension	Scenario	S^*	Static $/day	MDP $/day	Adv.
Cost	Base (c_u=2.50, c_o=1.50)	33	3.07	2.68	12.8%
	High margin (c_u=4.00)	35	3.78	3.40	10.0%
	Symmetric (c_u=c_o=1.50)	30	2.41	2.08	13.6%
	Expensive waste (c_o=3.00)	24	2.65	2.28	14.0%
Lead Time	Deterministic (P=1.0)	30	2.14	1.94	9.3%
	Base (P=0.8)	33	3.07	2.68	12.8%
	Noisier (P=0.6)	35	3.52	3.11	11.6%
	Coin flip (P=0.5)	35	3.62	3.24	10.4%
Shelf Life	2-day	28	6.40	5.10	20.3%
	3-day (base)	33	3.07	2.68	12.8%
	4-day*	37	1.61	—	≈0%

*MDP intractable at required capacity. Static S^* found via unconstrained grid search.

The Unifying Insight

Looking across all three sensitivity dimensions, a consistent pattern emerges. The MDP does not meaningfully improve the fill rate over the best static policy; it matches it. What it does instead is reduce waste. The static policy only has one knob to turn, S^*, and it uses that single knob to balance stockouts against waste. The MDP has access to the freshness breakdown of the current inventory, so it can maintain the same service level while discarding fewer sandwiches. This advantage is most pronounced when perishability is severe (short shelf life), when waste carries a high cost penalty, or when delivery times are moderately unpredictable.

VI. Validation

Monte Carlo convergence. All results are based on 1,000 replications of 364-day horizons. The cost differences between policies, for instance the $0.39 gap between the best static and MDP

policies, are many standard errors above zero, so the comparisons are statistically reliable.

Demand model. I validated the smoothed PMF by simulating demand paths from it and comparing them against the original historical histogram. The shapes matched closely. The pooled vs. weekday comparison described in Section II also provides out-of-sample evidence that the single pooled distribution generalizes well.

Value iteration. Across all parameter configurations tested, the MDP converged in about 21 iterations, with the Bellman residual decreasing monotonically each sweep and the number of policy changes dropping to zero. This was consistent across all 11 sensitivity scenarios (21–24 iterations in every case).

VII. Conclusions

This project went from raw café transaction data all the way to an optimized, state-dependent ordering policy for a perishable product. There are four main takeaways.

First, the standard textbook heuristics, ordering the mean and the newsvendor formula, performed very poorly here, achieving only around 40% fill rate. The core issue is that these formulas assume single-period, roughly symmetric demand with no perishability, and none of those assumptions hold in this setting. The right-skewed demand distribution is the main culprit.

Second, a well-chosen static order-up-to rule goes a long way. The optimal level $S^* = 33$ achieves a 94% fill rate at \$3.07/day, which is a 78% cost reduction over the mean policy. Most of the value comes from simply picking the right S.

Third, the MDP improves on this by another 13%, bringing cost down to \$2.68/day. The mechanism is waste reduction; it serves essentially the same number of customers but throws away fewer sandwiches by adapting orders to the freshness of what is currently on the shelf.

Fourth, the MDP's advantage is not universal. It is most valuable when shelf life is short (20% advantage at 2-day), waste is expensive (14%), or delivery times are moderately noisy (13%). If perishability is low, a static rule is sufficient and the MDP's added complexity is hard to justify.

Limitations. The demand model assumes stationarity, meaning no trends or seasonality. This held for the dataset used here, but may not generalize. The system also considers only a single product, whereas real cafés manage many items simultaneously. Finally, the cost parameters and lead time probabilities were assumed rather than estimated from data.

Future work. With a larger dataset, incorporating day-of-week demand effects could improve the model. Other extensions could also include real-time demand learning.

References

[1] Kaggle, "Coffee Shop Sales Dataset." Available: https://www.kaggle.com/. Accessed: Jan. 2026.

[2] S. Nahmias, "Perishable Inventory Theory: A Review," *Operations Research*, 30(4), 680–708, 1982.

[3] M. Khouja, "The single-period (news-vendor) problem: literature review," *Omega*, 27(5), 537–553, 1999.

[4] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, 4th ed., Athena Scientific, 2017.

[5] S. M. Ross, *Introduction to Probability Models*, 12th ed., Academic Press, 2019.

[6] Anthropic, "Claude" (AI assistant), 2024–2025. Used as a brainstorming and debugging collaborator during model development, including discussing MDP formulation choices, troubleshooting simulation logic, and exploring sensitivity analysis design. All modeling decisions, code implementation, and analysis are my own. https://claude.ai/.

Appendix A: Raw Data

The source dataset (`Cleaned_DataSet.csv`) contains 9,741 café transactions. Table 4 shows a representative sample. The full file is included as a supplementary material.

Table 4: Sample rows from the raw transaction dataset

Transaction ID	Item	Qty	Price	Total	Date
TXN_1961373	coffee	2	$2.00	$4.00	2023-09-08
TXN_4977031	cake	4	$3.00	$12.00	2023-05-16
TXN_6699534	sandwich	4	$4.00	$16.00	2023-10-28
TXN_2064365	sandwich	5	$4.00	$20.00	2023-12-31
TXN_3051279	sandwich	2	$4.00	$8.00	2023-02-06
TXN_7619095	sandwich	2	$4.00	$8.00	2023-05-03
TXN_3765707	sandwich	1	$4.00	$4.00	2023-06-10

After filtering for sandwiches and aggregating to daily totals, the cleaned dataset (`sandwich_daily_demand_cl` has 364 rows. Table 5 shows the first two weeks.

Table 5: First 14 days of cleaned daily demand

Date	Demand	Date	Demand
2023-01-01	11	2023-01-08	13
2023-01-02	16	2023-01-09	1
2023-01-03	6	2023-01-10	12
2023-01-04	13	2023-01-11	12
2023-01-05	20	2023-01-12	19
2023-01-06	12	2023-01-13	17
2023-01-07	2	2023-01-14	7

The smoothed demand PMF (`sandwich_P_of_D.csv`) is shown in Table 6. These are the probabilities used throughout all simulations and the MDP.

Table 6: Smoothed demand PMF ($\alpha = 1.0$ Dirichlet smoothing)

D	$P(D)$	D	$P(D)$	D	$P(D)$
0	0.0436	9	0.0667	18	0.0231
1	0.0333	10	0.0410	19	0.0154
2	0.0359	11	0.0564	20	0.0179
3	0.0641	12	0.0513	21	0.0128
4	0.0436	13	0.0282	22	0.0103
5	0.0641	14	0.0385	23	0.0077
6	0.0769	15	0.0308	24	0.0077
7	0.0692	16	0.0308	25	0.0051
8	0.0923	17	0.0333		

Appendix B: Code

Full source code is in notebooks `01_data_cleaning.ipynb` through `06b_sensitivity.ipynb`. The three core components are shown below.

B.1: Vectorized Monte Carlo Simulator

This function evaluates a static order-up-to-S policy by running 1,000 parallel simulations of 364 days each. All runs execute simultaneously through NumPy array operations.

```
def simulate_static(S, shelf_life, p_lead1, cu, co,
                    n_runs=1000):
    rng = np.random.default_rng(42)
    E = np.zeros((shelf_life, n_runs))  # age buckets: E[0]=oldest
    A1 = np.zeros(n_runs); A2 = np.zeros(n_runs)  # pipeline
    tu = tw = td = np.zeros(n_runs)     # accumulators

    for t in range(364):
        # Step 1: Receive deliveries (pipeline A1 arrives)
        arr = A1.copy(); A1[:] = A2; A2[:] = 0
        E[shelf_life-1] += arr          # arrivals = freshest bucket

        # Step 2: Compute inventory position and order
        ip = E.sum(axis=0) + A1 + A2    # on-hand + pipeline
        q = np.maximum(0, S - ip)        # order up to S

        # Step 3: Assign lead time to new orders
        m = q > 0
        if m.any():
            L = np.where(rng.random(n_runs) < p_lead1, 1, 2)
            A1 += np.where(m & (L==1), q, 0)
            A2 += np.where(m & (L==2), q, 0)

        # Step 4: Realize demand, serve FIFO (oldest first)
        D = rng.choice(demand_values, n_runs, p=demand_probs)
        td += D; rem = D.copy()
        for i in range(shelf_life):
            x = np.minimum(E[i], rem); E[i] -= x; rem -= x

        # Step 5: Record unmet demand, discard expired, age
        tu += rem; tw += E[0]; E[0] = 0
        for i in range(shelf_life-1):
            E[i] = E[i+1].copy()
        E[shelf_life-1] = 0

    return (cu*tu + co*tw) / 364         # avg daily cost per run
```

B.2: Grid Search over Order-Up-To Level

This loop calls the simulator at each candidate S value and records the results for comparison.

```
# Sweep S from 1 to 50, evaluate each with 1000 MC runs
```

```
results = []
for S in range(1, 51):
    costs = simulate_static(S, shelf_life=3, p_lead1=0.8,
                            cu=2.50, co=1.50, n_runs=1000)
    results.append({
        'S': S,
        'mean_cost': costs.mean(),
        'p5': np.percentile(costs, 5),
        'p95': np.percentile(costs, 95)
    })

df = pd.DataFrame(results)
S_star = df.loc[df['mean_cost'].idxmin(), 'S']
# Result: S* = 33 at $3.07/day
```

B.3: C Bellman Update (MDP Value Iteration)

The inner loop of the MDP solver, written in C for performance. Tight loop bounds ensure only feasible states are visited (about 82,000 out of a potential 1.68 million).

```
/* C Bellman update -- tight loop bounds, only feasible states */
void vi_sweep_M3(double *V, double *V_new, int *policy,
    int *dvals, double *dprobs, int n_d,
    double cu, double co, double p1, double gamma, int cap,
    double *max_diff) {
  int dim = cap + 1;
  for (int e0=0; e0<dim; e0++)
  for (int e1=0; e1<dim-e0; e1++)       /* tight bounds */
  for (int e2=0; e2<dim-e0-e1; e2++)
  for (int a1=0; a1<dim-e0-e1-e2; a1++) {
    /* For each feasible state (e0,e1,e2,a1):
       - Try all order quantities q = 0..cap-e0-e1-e2-a1
       - For each q, loop over demand d and lead time L
       - Compute immediate cost + discounted future value
       - Pick q that minimizes expected total cost
       - Store optimal q in policy[], new value in V_new[] */
    /* ... (full implementation in notebook 06_mdp.ipynb) ... */
  }
}
```